

ons. Modern infrastructure needs to be elastic. Capacity that can be dynamically provisioned and de-provisioned is able to keep pace with load maintaining optimal performance and minimal cost. With the demands made on today's infrastructure it's becoming increasingly crucial manage infrastructure automation with a robust and cohesive methodology.

What is GitOps?

text

GitOps == IaC + MRs + CI/CD

> GitOps is an operational framework that takes DevOps best practices used for application development such as version control, collaboration, compliance, and CI/CD, and applies them to infrastructure automation.

<iframe width="100%" height="500" src="https://www.youtube.com/embed/JtZfnrwOOAw" frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

GitOps involves managing your IT infrastructure using practices well-known in software development such as version control, code review, and CI/CD pipelines. For example, infrastructure teams that practice GitOps use configuration files stored as code. Similar to how application source code generates the same application binaries every time it's built, GitOps configuration generates the same infrastructure environment every time it is deployed.

- IaC - GitOps uses a Git repository as the single source of truth for infrastructure definition. Infrastructure as Code (IaC) is the practice of keeping all infrastructure configuration stored as code. The actual desired state may or may not be not stored as code (e.g., number of replicas, pods).
- MRs - GitOps uses Merge Requests (MRs) as the change mechanism for all infrastructure updates. The MR is where teams can collaborate via reviews and comments and where formal approvals take place. Merge commits to your master(or trunk) branch serve as a change log for auditing and troubleshooting.
- CI/CD - GitOps automates infrastructure updates using a Git workflow with Continuous Integration and Continuous Delivery (CI/CD). When new code is merged, the CI/CD pipeline enacts the change in the environment. Any configuration drift, such as manual changes or errors, is overwritten by GitOps automation so the environment converges on the desired state defined in Git. GitLab uses CI/CD pipelines to manage and implement GitOps automation, but other forms of automation such as definitions operators could be used as well.

As with any emerging technology term, "GitOps" isn't strictly defined the same way by everyone across the industry. GitOps emerged in the cloud native community and some definitions restrict GitOps to say "Kubernetes is required to be doing GitOps." GitLab takes a broader approach. We've seen GitLab users and customers applying GitOps principals to all types of infrastructure automation including VMs and containers, as well as Kubernetes clusters.

While many tools and methodologies promise faster deployment and seamless management between code and infrastructure, GitOps differs by focusing on a developer-centric experience. Infrastructure management through GitOps happens in the same version control system as the application development, enabling teams to collaborate more in a central location while

benefiting from all the built-in features of Git.

GitOps is a prescriptive workflow for using Infrastructure as Code. GitOps with GitLab helps you manage physical, virtual and cloud native infrastructures (including Kubernetes and serverless technologies) using tight integration with industry-leading infrastructure automation tools like Terraform, AWS Cloud Formation, Ansible, Chef, Puppet, and the like.

Benefits of GitOps

- Distribution of work - Enable more engineers to collaborate on infrastructure changes. Since every change will go through the same change/merge request/review/approval/merge process, this enables senior engineers to focus on other areas beyond the critical infrastructure management while still maintaining the ability to review and contribute as needed.
- Improved access control There's no need to give credentials to all infrastructure components since changes are automated only by your CI/CD needs access.
- Faster time to market - execution via code is faster than manual point and click, test cases can be automated and hence repeatable in a consistent manner to deliver stable environments rapidly, at scale
- Less risk The shift-left approach to infrastructure as Code helps to identify and resolve issues before changes are rolled out to production preempting unexpected downtime and providing higher environment stability and reliability and better user experience.
- More compliant All changes to infrastructure are tracked, leaving traceability for audit and ability to rollback to a previous desired state with ease.
- Reduce costs - automation of infrastructure definition and testing eliminates manual tasks and rework improving productivity, reduced downtimes due to built in revert/rollback capability
- Less error prone - infrastructure definition is codified, hence repeatable and less prone to human error

Push vs Pull GitOps (Agentless vs Agent Based GitOps)

As with any emerging technologies, there are different approaches to GitOps, each with their own pros and cons.

Push or Agentless GitOps

In this approach, your CI/CD tool pushes the changes to your environment. This approach is consistent with the approach used for application deployment.

Pro

- Ease of use. Well-known CI/CD - build, test, & deploy all use the same tech
- Deployment targets not limited to cloud native / Kubernetes only - can deploy to physical, virtual container - whether on premise or cloud etc.

Con

- Need to open your firewall to your cluster and grant admin access to external CI/CD

Pull or Agent Based GitOps

In this approach, an agent is installed in your cluster to pull changes whenever there is a drift from the desired configuration.

Pro

- Secure infrastructure - no need to open your firewall or grant admin access externally

Con

- Agent needs to be installed in every cluster
- Limited to k8s-only
- Uses different technology than application CI/CD

Personas

User Persona

Infrastructure as Code requires understanding of the platform and the desired state of the application environment. Users of Infrastructure as Code have a good understanding of both Git as a SCM tool as well as the platform they are expected to provision and manage. Below are a few power users Infrastructure as Code:

- Sam, the SRE

Sam works embedded into a service team with feature developers. They works on keeping the service up, deploying it, and managing the infrastructure needs of that service. They collaborate with the Platform team to systematize best practices.

- Devon, the DevOps Engineer

Devon is often the Ops interface for the development team. He provides support for infrastructure, environments and integrations. Devon is fairly conversant with code and would prefer administering infrastructure via code rather than a multitude of different tools and context switches.

- Priyanka, the Platform Operator

Infrastructure management is one of the main responsibilities of the platform team. Priyanka is responsible for providing, maintaining, and operating a shared platform - either traditional or modern cloud platform - which the development teams utilize to ship and operate software more quickly.

- Sydney, the System Administrator

Sydney defines, maintains and scales the infrastructure and configuration for the application teams. She often receives repetitive requests on the same task. Sydney's primary motivation is to automate repetitive tasks to minimize errors and save time as well as define the infrastructure and configuration in a way that changes are tracked and to stop infrastructure changes becoming the wild west.

Buyer Personas

Buyers of Infrastructure as Code are usually leaders who lead infrastructure / automation initiatives. Typical buyer personas are:

- CIO / Vice President of IT - experience in planning, design and execution of digital transformation programs, implementing new operating models. Typically leads both engineering and operations teams
- Vice President of IT Infrastructure (also referred to as IT Operations in some organizations)
- experience in planning, design and execution of IT infrastructure services - including deploying and managing cloud services, system management and Service Desk. Frequently has the agenda of reducing IT costs for the organization.

- Vice President of Platform Engineering - Managing a shared platform for development teams is one of the main agendas of the Platform Engineering team. The platform team has expertise in new technologies like Kubernetes. Key KPIs of the platform engineering team are Automation, Efficiency and Self Service.

Analyst Coverage

List key analyst coverage of this use case

Market Requirements

Below are the market requirements for GitOps

Foster Collaboration

- Description: The solution is designed to enable and foster collaboration among team members. The collaboration system includes manual gates and approvals as well as automated workflows.
- Typical features:
 - Add a general comment
 - Tag individuals or groups
 - Add an inline comment on code
 - Add a suggestion
 - Show the total unresolved threads at the tops
 - Show you can't merge until you resolve comments/suggestions
 - Resolve a thread
 - Resolve a thread by creating a new issue
 - Apply a suggestion (accept a suggestion)
 - Show codeowners
 - Show approval
- Other features
 - Quickly create new branches of a project
 - Add new files/assets
 - Collaborate on proposed changes (review comments, suggest changes, WebIDE, suggestion approvals, conflict resolution, merge, diffing, hand-offs)
 - Workflow automation
 - Wiki snippets
 - Version-controlled snippets
 - Automatically update or close related issue(s) when a merge request is merged
 - Configurable issue closing pattern
 - Display merge request status for builds in CI system
 - Terraform plan output
 - Visibility into security scans and build stats
- Value: Quality of the code changes being made increases which leads to
 - greater accuracy of fulfilled requests
 - greater infrastructure stability
 - and improved release velocity through team review and validation.

Compliance and Auditing

- Description: All changes to production are automated and gated by merging to a Git branch. Permission and access are built-in via the Git management tool.
 - Approvals and merge access can be restricted to certain personnel or groups.
 - Commit history shows the log of changes.
- Typical Features: CMRs/PRs, code review features, RBAC, branch permissions, group permission setting, CodeOwners, MR approvals
- Value: Don't have to manage separate permissions for your infrastructure; they can match that of your development process.
 - Strong adherence to internal and regulatory compliance standards.
 - Audits take less time and are easier to conduct.
 - Changes to infra are more secure because they are locked down.

\

Version controlled environments

- Description: Ability to roll back and roll forward your environments when their state is defined via text stored under Git version control.

Test Automation

- Description: Run and manage automated tests and validate changes before they're merged to production. This includes everything from basic to more in-depth tests and extends test automation into areas of functional, system, performance testing, and more. Ensure software is consistently tested to meet both technical and business requirements without manual intervention, enabling developers to get rapid feedback if their code changes introduce potential defects or vulnerabilities.
- Typical Features: The ability to run automated tests in CI/CD pipelines in isolated and ephemeral environments. Various tests may include (but aren't limited to) unit testing, code integration testing, regression testing, static code analysis, functional testing, and accessibility testing. Most types of testing for app code also apply to any type of code (infra, policy, etc.).
- Value: Catch potential errors sooner, rather than later- before they impact production. Increase security by testing for potential vulnerabilities, before going to production.

Pipeline configuration management

- Description: Engineers can automate the build and test workflow, specifically connecting to their source code repository, defining specific actions/tasks in build pipelines, and set parameters for exactly how/when to run jobs, using scripts and/or a GUI to configure pipeline changes. Configurations are repeatable and traceable to allow for quick comparisons and tracking of changes to environments.
- Typical Features: Configurations via web UI or supports config-as-code in a human readable syntax, like YAML.
 - Pipeline templates. Project Templates.
 - Pipeline linking - store pipeline template in central repo
 - Pipeline dependency - child/parent, upstream/downstream
- Value: Maximize development time and improves productivity. Less manual work.
 - Centralize management to lower administration.

Increase consistency by using the templates to adhere to best practices and required testing.

The GitLab Solution

How GitLab Meets the Market Requirements

A collection of short demonstrations that show GitLab's GitOps capabilities.

| Market Requirements | How GitLab Delivers | GitLab Stage/Category | Demos |
| ----- | ----- | ----- | ----- | ----
| Foster Collaboration | Discussions, user tagging, general comments, inline comments, inline suggestions, unresolved thread tracking, issue creation from comment, suggestion management, CODEOWNERS, approvals | Discussions, CODEOWNERS, MR approvals | Foster Collaboration with GitOps |
| Compliance and Auditing | Compliance testing and audit controls are built into GitLab's CI/CD pipelines. | Compliance at GitLab
 Manage Stage: Audit Events, Audit Logs, Audit Reports, Compliance Management, Release Evidence, Chain of Custody, Granular user roles and permissions, segregation of duties
 Secure Stage: License Compliance, Dependency Scanning
 CODEOWNERS, MR approvals, | |
| Version controlled environments | Using GitLab version control to store XaC (Infrastructure, policy, configuration etc) helps maintain track of changes. In addition, auto rollback in case of failure helps roll back to the previous successful deployment | Environment Rollback, GitLab Agent for Kubernetes | |
| Test Automation | Include usability, performance testing as a part of the pipeline and roll back to a successful deployment in case of issues | Release versioning, support advanced deployment strategies such as canary, incremental roll out, blue green deployments, Feature Flags, Review Apps, Performance testing & validation | |
| Pipeline configuration Management | GitLab Pipeline Authoring allows users to create pipelines with minimal configuration, if required, and also create and visualize complex pipelines. This helps infrastructure engineers minimize manual work and create repeatable processes to maximize productivity | Pipeline configuration, Types of Pipeline configurations |
|

Top 3 Differentiators

Differentiator	Value	Proof Point
Choice of Agent-based and Agentless approach	Customers have a choice to pick the right approach for their environments. With an Agentless approach, users can have GitLab manage and orchestrate GitOps processes for K8s and non-K8s platforms without having to install and maintain agents at every destination infrastructure component leading to the consumption of less resources, and cost savings. With an Agent-based approach, users, with secured infrastructure components that sit behind a firewall, can leverage the GitLab Agent to keep their secured infrastructure up-to-date mitigating the risk of security breaches and unexpected downtimes.	TBD
On-premise & cloud - physical, virtual, cloud native infrastructures	GitLab meets customers where they are. Most competitors support GitOps for cloud native infrastructure only, GitLab supports GitOps for your on-prem or cloud deployments - whether it is physical servers, virtual servers or cloud native infrastructures. You don't need to purchase and maintain	

different GitOps solutions for your different kinds of infrastructure leading to cost savings and a consistent user experience for doing GitOps across your organization. | TBD |
| Single application to foster collaboration across Infrastructure, Development and Operations team | GitLab is a single application with most of the necessary ingredients for GitOps - version control, CI/CD and container registry & out-of-the-box integration for configuration management, orchestration & infrastructure provisioning. Most competitive solutions require 5-6 integrations to achieve the same. Consolidating into GitLab for all your GitOps leads to cost savings and its consistent user experience and integrated capabilities can help increase productivity and higher environment stability and reliability. | TBD |

Message House

The message house for the use case provides a structure to describe and discuss the value and differentiators for the use case.

Current messaging can be found in the market viewpoint section for this page and on the topic page which answers What is GitOps? along with some additional notes here:

"Infrastructure Platform" is a great phrase to use in connection with GitOps and GitLab. We need to be careful with its use so we don't portray ourselves as something we aren't. In the past we've been called out for claiming to have capabilities similar to Chef, Ansible, and Terraform, when we don't house the capabilities within GitLab, but instead seek to integrate with those tools. If we keep it clear that we provide infra platform capabilities via integrations then we have some strong messaging that connects with I&O buyers. See an example use on the topic page

Discovery Questions

- How does your infrastructure team manage configurations, policies, variables etc consistently across your infrastructure?
- How are you able to keep track of the changes made to your infrastructure?
- Are you able to ensure that changes are reviewed and approved by the right individuals - to ensure minimal disruption to your staging / production environments?
- How many people have access to your environments console and are you able to track changes made in the console consistently? e.g., engineer logs into AWS / GCP console to make changes to create a break fix solution.
- How are you able to showcase proof of compliance of all the infrastructure changes done - that are essential audit requirements?
- How quickly are you able to recover from issues in your environment and roll back to the last successful deployment?
- Is your infrastructure team able to consistently repeat the steps to setup an environment every single time?
- Are your infrastructure teams and development teams using different tools - thereby reducing collaboration between the two teams?
- Are you able to consistently deploy your infrastructure policies and configurations to multiple clouds to ensure continuity across clouds and minimal tooling?
- We've heard some customers struggle with configuration files being stored in an ad-hoc manner. How are you managing your configuration files?
- Many customers have more than one infrastructure administrators managing their environment. How are you able to track the root cause of your production downtime when multiple people have

access to make changes to the environment?

- How manual and repetitive is your infrastructure setup and maintenance activities?
- How much time do your engineers spend doing infrastructure setup and maintenance? How much of it is repetitive?
- How satisfied are your engineers with their workflow? What areas would they like to improve?
- How much productivity is lost due to manual changes to your environments?
- Is your organization interested in improving infrastructure workflows in favor of more automation?
- Why do you believe you need infrastructure automation in your organization?
- What type of roadblocks are you encountering with your current infrastructure workflows?
- What is the workflow for an infra engineer to make any changes to the current staging / production environments?
- How much time does it currently take to setup new infrastructure workflows? How repetitive are these workflows?

Objection Handling

| Objection | Response |

| ----- | ----- |

| Our environment is too complex for GitOps | Large changes are always complex. But you can consider doing this iteratively - starting from your simplest test environment. GitLab also has a strong professional services team that can help you get started with your GitOps journey. |

| GitOps gives more access to developers to fiddle with the deployments, and infrastructure teams are not comfortable with that | GitOps empowers developers to work collaboratively with Infrastructure teams. GitOps with GitLab includes enough controls through merge request approvals, audit reports, compliance dashboards - that your infrastructure teams can use to ensure they are able to monitor what goes into production. |

| (Infra / DevOps Engineer) I'm going to loose control over my work and environment | GitOps empowers developers to work collaboratively with Infrastructure teams. GitOps with GitLab includes enough controls through merge request approvals, audit reports, compliance dashboards - that your infrastructure teams can use to ensure they are able to monitor what goes into production. |

| I don't use Kubernetes, so GitOps is not relevant for me | GitOps with GitLab will meet you where you are - we support deployments to physical, virtual and cloud native environments - whether on-premise or on cloud. |

| I am not comfortable with a push-based / agentless approach that expects me to open up my firewall or give cluster access to GitLab CI/CD. | We recently announced a GitLab Agent for Kubernetes that would allow secure communication between the Kubernetes cluster and GitLab CI/CD using an agent-based approach. This would not require customers to open up their firewall or give cluster access to GitLab CI/CD. We support both options currently - and you can choose the approach that best suits your specific needs. |

Competitive Comparison

ArgoCD

ArgoCD is an Open Source Solution for GitOps deployments for Kubernetes. It was started by Intuit and is now a CNCF incubation project. It is part of the Argo Project family of products.

ArgoCD is competes directly with the GitLab Agent for Kubernetes.

ArgoCD works together with GitLab. It can supplement the build and secure steps run in GitLab CI/CD.

| Differentiator | How competitor does it | How GitLab does it differently |

| - | ----- | ----- |

| Multiple products to achieve GitOps | ArgoCD focuses exclusively on the deployment part.

There are other Argo projects for feature flags and advanced deployments strategy support. |

GitLab provides all the required tooling integrated in a single platform |

| Multiple deployment targets | Focused on cloud native deployments only | Can deploy to On-premise & cloud, physical, virtual, cloud native infrastructures |

| Push & Pull based deployment Automation | ArgoCD watches for new images and updates services in the K8S cluster according to policies chosen by the customer. Does not support push based deployment. | The GitLab Agent for Kubernetes watches for new K8S manifests and updates the K8S cluster accordingly. |

| Use GitOps principles to install and manage self | ArgoCD can bootstrap itself | The GitLab Agent can set up to manage itself |

| Synchronize state when deployment is rolled back | As a rollback is a new git commit or revert, this happens by design | As a rollback is a new git commit or revert, this happens by design when using the GitLab Agent |

Weaveworks - using Flux OSS operator

Flux is an Open Source Solution for continuous and progressive delivery for Kubernetes. Flux was started at Weaveworks and is now a CNCF incubation project. It is part of the WeaveWorks Cloud solution for GitOps.

The WeaveWorks Cloud solution also works together with GitLab - with GitLab SCM and CI working with Flux CD to create a GitOps solution.

| Differentiator | How competitor does it | How GitLab does it differently |

| - | ----- | ----- |

| Multiple products to achieve GitOps | The Weaveworks toolchain is GitHub for version control, CircleCI for CI , Quay.io for container registry, Weave Flux for CD in addition to other tools for configuration management, infrastructure provisioning and container orchestration Source.

Customers can choose any other Git, CI, container registry of their choice as well | Customers have at least 3 fewer tools to manage. Version control, CI/CD and container registry is offered by GitLab while we integrate with other tools for configuration management, infrastructure provisioning and container orchestration similar to Weaveworks. |

| Multiple deployment targets | Focused on cloud native deployments only | Can deploy to On-premise & cloud, physical, virtual, cloud native infrastructures |

| Push & Pull based deployment Automation | Flux watches for new images and updates services in the K8S cluster according to policies chosen by the customer. Does not support push based deployment. | The GitLab Agent for Kubernetes watches for new K8S manifests and updates the K8S cluster accordingly. Available on self managed and on invite only for SaaS. Currently - push based deployment is supported without agent. Roadmap includes push based deployment support as well using the agent. |

| Use GitOps principles to install and manage self | Flux v2 can create a git repo, manage flux manifests and updates using GitOps principles | TBD? |

| Synchronize state when deployment is rolled back | TBD? | TBD? |

Industry Analyst Relations (IAR) Plan

- The IAR Handbook page has been updated to reflect our plans for incorporating Use Cases into our analyst conversations.
- For details specific to each use case, and in respect of our contractual confidentiality agreements with Industry Analyst firms, our engagement plans are available to GitLab team members in the following protected document: IAR Use Case Profile and Engagement Plan.

For a list of analysts with a current understanding of GitLab's capabilities for this use case, please reach out to Analyst Relations via Slack (analyst-relations) or by submitting an issue and selecting the "AR-Analyst-Validation" template.

Adoption recommendation

This table shows the recommended use cases to adopt, links to product documentation, the respective subscription tier for the use case, and product analytics metrics.

Feature / Use Case	Free	Premium	Ultimate	Product Analytics	Notes
-----	----	-----	-----	-----	-----
Try the GitLab Agent for Kubernetes	·	·	·	counts.kubernetesagentswithtoken	
Migrate from custom CI solution	·	·	·	·	·
redishllcounters.kubernetesagent.agentusersusingcitunnelmonthly					
Use push based deployments	·	·	·	·	·
redishllcounters.kubernetesagent.agentusersusingcitunnelmonthly					
Advanced permissions management for CI connection	·	·	·	·	·
redishllcounters.kubernetesagent.agentusersusingcitunnelmonthly					
Use pull-based deployments	·	·	·	·	WIP

Proof Points - customers

Quotes and reviews

- Video: kiwi.com on Infrastructure as Code - Learn how kiwi.com uses GitLab and Terraform to manage their infrastructure as code. Bonus info - see how they deploy self hosted GitLab instance from GitLab CI/CD and Infrastructure as Code
- Video: VMWare - Infrastructure as Code with GitLab and Terraform Cloud - GitLab and Terraform have worked well together for some time; Learn how VMWare uses GitLab CI/CD and Terraform Cloud to manage infrastructure as code
- Video: ValidaTek's journey of DevOps and Infrastructure as code - ValidaTek work with all areas of the US Federal government from civilian to military to IC. See how they's set up GitLab to manage their client's infrastructure.

Gartner Peer Insights

"Very efficient tool for managing releases and versions. We have a development and deployment process, and at all stages [GitLab] is involved. In addition to storing development code, we also store all packaging and deploy scripts in git"

>

> - Full-stack Developer, Gartner Peer Insights Review

"Finally, the most amazing thing about GitLab is how well integrated the GitLab ecosystem is. It covers almost every step of development nicely, from the VCS, to CI, and deployment."

>

> - Software Engineer, Gartner Peer Insights Review

"GitLab is the most preferred service in the world and its user community is very wide. We can authorize project or branch based user authorization on GitLab. In addition, continuous deployment integrations can be done very quickly. In addition, you can create merge requests within the constraints you want and easily manage them. It is very easy to prevent conflicts. A service that must be used for software development teams."

>

>- Software Development Lead, Gartner Peer Insights Review

Case Studies

-Northwestern Mutual

- Problem Manual processes, provisioning of resources from On-Prem Datacenters created inefficient practices.
- Solution GitLab Premium (SCM,CI) and Terraform
- Result Drastically improved lead time to production, Environments can be created in under an hour. Everything is done as code and there are no risks from patching.
- Sales Segment: Enterprise

-Wag Labs

- Problem Lack of control complex CI toolchain.
- Solution GitLab Ultimate (SCM,CI,CD) and Terraform
- Result It's so easy to deploy something and roll it back if there's an issue. It's taken the stress and the fear out of deploying into production.
- Sales Segment: Mid-Market

References to help you close

- Find customers that are using GitLab with GitOps

Partners

GitLab is not a replacement for existing Infrastructure Automation tools, but rather complements them to provide a comprehensive solution. As per the JetBrains DevOps Ecosystem 2019 survey, Terraform is the most popular infrastructure provisioning tool used by customers. Terraform is cloud-agnostic and helps manage complex infrastructures for distributed applications. GitLab will focus on Terraform support as the first step towards building a comprehensive GitOps solution.

Resources

Presentations

- GitOps Industry Talk - What is GitOps? Why is it important? How can you get started? - slide

deck, video

WebPage, Whitepapers, infographics, blogs

- What is GitOps?
- Infrastructure as Code using GitLab & Ansible
- Part 1 of 3: Why collaboration technology is critical for GitOps
- Part 2 of 3: How infrastructure teams use GitLab and Terraform for GitOps
- Part 3 of 3: How to deploy to any cloud using GitLab for GitOps

Videos (including basic demo videos)

- What is GitOps? Why is it important? How can you get started?
- What is Infrastructure as Code
- Infrastructure as Code using GitLab & Ansible
- Part 1 of 3: How GitLab supports GitOps: The Process
- Part 2 of 3: How GitLab supports GitOps: The Infrastructure
- Part 3 of 3: How GitLab supports GitOps: The Application
- GitOps with GitLab and Terraform
- Using GitLab for GitOps to break down silos and encourage collaboration

Clickthrough & Live Demos

- GitOps Click Through Demo

Buyer's Journey

Inventory of key pages in the buyer's Journey

| Awareness
 learning about the problem | Consideration
 looking for solution ideas
| Decision
 is this the right solution|
| ----- | ----- |----- |
| topic page? | solution page | proof points |
| landing pages? | ?comparisons? | comparisons |
| -etc? | | - product page x
 - product page y
 - product page z |

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/gitops/keywords.md

SEO and Keywords for USECASENAME

- Keyword
- etc.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/gitops/message-house-template.md

| Positioning Statement: | (how GitLab fits and is differentiated in the market for this use case) |

|-----|-----|

| User Persona | a few words and a link to the details |

| Buyer Persona | a short description and a link to details |

| Short Description | 25 words or less |

| Long Description | 100 words or less |

| Key-Values | Value 1: <List a key message/ value proposition> | Value 2: | Value 3: |

|-----|-----|-----|

-----|

| Promise | (list and describe the positive business outcomes) | | |

| Pain points | (describe common pain points) | | |

| Why GitLab | (list specific features that support this value) | | |

| Proof points | (list specific analyst reports, case studies, testimonials, etc) |

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/template/_index.md

markdown

<!--

-->

Who to contact

| PMM | TMM |

| ---- | --- |

| xyz | abc |

The Market Viewpoint

UseCase Name

Define and describe the use case in customer terms. Reference the customer use case page

- Keywords and SEO Terms

Personas

User Persona

Who are the users (a few remarks and link to the persona page)

Buyer Personas

Who are the buyers

Industry Analyst Resources

List key analyst coverage of this use case

Market Requirements

Requirement	Description	Typical features	Value
-----	-----	-----	-----
abc	def	ghi	jkl

The GitLab Solution

How GitLab Meets the Market Requirements

Market Requirements	How GitLab Delivers	GitLab Category	Demos
-----	-----	-----	-----
REQMT	HOW	CATS	DEMO

Top 3 Differentiators and Key Features

Differentiator	Value	Proof Point
-----	-----	-----
abc	def	ghi

Message House

The message house for the use case provides a structure to describe and discuss the value and differentiators for the use case.

Customer Facing Slides

Discovery Questions

- list key discovery questions

Competitive Comparison

TBD - will be a comparison grid leveraging the capabilities

Industry Analyst Relations (IAR) Plan (UPDATE AS NEEDED FOR THIS USECASE)

- The IAR Handbook page has been updated to reflect our plans for incorporating Use Cases into our analyst conversations.
- For details specific to each use case, and in respect of our contractual confidentiality agreements with Industry Analyst firms, our engagement plans are available to GitLab team members in the following protected document: IAR Use Case Profile and Engagement Plan.

For a list of analysts with a current understanding of GitLab's capabilities for this use case,

please reach out to Analyst Relations via Slack (analyst-relations) or by submitting an issue.

Proof Points - customers

Quotes and reviews

- List of customer quotes/reviews from public sites

Case Studies

- List of case studies

References to help you close

- Link to SFDC list of use case specific references

Adoption Guide

Playbook Steps

- To be added...

Adoption Recommendation

This table shows the recommended use cases to adopt, links to product documentation, the respective subscription tier for the use case, and product metrics.

Feature / Use Case	F/C	Basic	S/P	G/U	Notes	Telemetry
-----	:-----:	:-----:	:-----:	:--:	:-----:	
example	example	example	example	example	example	example

Enablement and Training

Professional Service Offers

Partners

- Describe how key partners help enable this use case

Key Value (at tiers)

Premium

- Describe the value proposition of why Premium for this UseCase

Ultimate

- Describe the value proposition of why Ultimate for this UseCase

Direction

- Describe a vision for compelling future improvements

Resources

Presentations

- LINK

Whitepapers and infographics

- LINK

Videos (including basic demo videos)

- LINK

Integrations Demo Videos

- LINK

Clickthrough & Live Demos

- Link

Buyer's Journey

Inventory of key pages in the buyer's Journey

| Awareness
 learning about the problem | Consideration
 looking for solution ideas
| Decision
 is this the right solution|
| ----- | ----- |----- |
| topic page? | solution page | proof points |
| landing pages? | ?comparisons? | comparisons |
| -etc? | | - product page x
 - product page y
 - product page z |

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/template/keywords.md

SEO and Keywords for USECASENAME

- Keyword
- etc.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/template/message-house-template.md

| Positioning Statement: | (how GitLab fits and is differentiated in the market for this use case) |

|-----|-----|
----|

| User Persona | a few words and a link to the details |

| Buyer Persona | a short description and a link to details |

| Short Description | 25 words or less |

| Long Description | 100 words or less |

| Key-Values | Value 1: <List a key message/ value proposition> | Value 2: | Value 3: |

|-----|-----|-----|
-----|

| Promise | (list and describe the positive business outcomes) | | |

| Pain points | (describe common pain points) | | |

| Why GitLab | (list specific features that support this value) | | |

| Proof points | (list specific analyst reports, case studies, testimonials, etc) |

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/version-control-collaboration/_index.md

Looking for an overview of GitLab's Source Code Management(SCM) capabilities?

See the SCM Solution.

The page below is intended to align GitLab sales and marketing efforts with a single source of truth for our go-to-market efforts around Source Code Management.

Who to contact

| Product Marketing | Developer Advocate |

| --- | --- |

| Aathira Nair (@anair5) | William Galindez Arias |

Software Delivery Automation

Software Delivery Automation allows organizations to automate manual, repetitive tasks from their SDLC to improve the overall velocity of the software factory and increase collaboration across dev and ops. This includes both application and infrastructure automation.

From a GitLab capabilities point of view - it includes DevOps essentials like SCM, CI, CD, GitOps and AutoDevOps.

Source Code Management is one part of the Software Delivery Automation solution. See how Source Code Management fits into the overall Software Delivery Automation solution.

The Market Viewpoint

Source Code Management (SCM)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/TxfAeYXmles"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>

As organizations accelerate delivery, through DevOps, controlling and managing different versions of the application assets from code to configuration and from design to deployment is incredibly important.

Velocity without robust version control and traceability is like driving a car with out a seatbelt.

Source Code Management is more than simply tracking changes, versions, and branches of code. Effectively, it includes practices such as:

- Enabling development teams to work in distributed and asynchronous environments
- Managing changes and versions of code and artifacts
- Enabling Review and Collaboration of code and other assets
- Tracking approvals of proposed changes
- Resolving merge conflicts and related anomalies

In general, version control is required because software is constantly changing. Regardless of the stage of development, there will be change to deal with.

> No matter where we are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle.

>

> E.H. Bersoff, 1980.

Companies that excel at source code management create high quality code while integrating it at high frequency.

Personas

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/2ghFIR5DLbo"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>

User Persona

Being the entry point to GitLab means that many user personas find utility and a solution to their problem in Source Code Management. Let's go through the list of power user personas and describe briefly their key motivations to use Source Code Management in GitLab:

Parker the Product Manager

- PMs coordinate feature development and project success among other things. The ability to monitor progress through commits, review app and validate those changes and provide feedback is key to their success of their role
- These changes, in time will generate valuable statistical insight for the PM to assess accurately development efforts of planned new features

Delaney the Development Team Lead

- Just like the PM, Team Leads need also to understand their team's capacity to assign upcoming tasks to meet goals on time accordingly
- Approval workflows in Code Review allows them to become faster real team work enablers

Sasha the Software Developer

- Sasha takes advantage of both Command Line Tools and GitLab's GUI to have complete control of every commit he does to complete his tasks
- Even when scope changes, a frequent hurdle and a source of frustration, branching, merging and conflict resolution will be performed in Source Code Management and will trigger CI for fast resolution

Devon the DevOps Engineer

- All information relevant to the role's goals is congregated in Source Code Management to take action on it. Time to resolution and, in general, any other key metric in DevOps is measured and its performance tracked in Source Code Management
- Any improvement applied to the development process will reflect in Source Code Management's interfaces, whether its the Merge Requests or Issues.

Cameron the Compliance Manager

- Cameron needs to all the company's development processes are compliant. Given the amount of data that a software development and delivery lifecycle produces, he finds it difficult to find, aggregate, and report on all of the necessary data for audit purposes
- He needs the information to be available quickly and easily so he can reduce the time and disruption involved in the evidence collection process

Presley the Product Designer

- Presley is responsible for understanding user needs and product requirements to create and iterate on design proposals to solve user needs
- He would like to have clear, up-to-date requirements that he can reliably refer back to throughout the design process

Sydney the Systems Administrator

- Sydney is in charge of making tools such as GitLab available and accessible to everyone.
- She needs to understand in depth the provision and scalability of the infrastructure to provide the company with a solid, fault tolerant instance of the platform.

Buyer Personas

Source Code Management purchasing typically do not require executive involvement. It is usually acquired and installed via our freemium offering without procurement or IT's approval. This process is commonly known as shadow IT and its a great opportunity for us to eventually become a paid for service. When the upgrade is required the VP of IT is the most frequent decision maker. The influence of the VP Application Development is notable too to the owner of the budget.

Alex the Application Development Manager

- Alex is highly technical, and only one step removed from an individual contributor.
- Managing his budget and ensuring that his team has the right skills and overall team engagement.

Dakota the Application Development Director

- Dakota is a key IT leader who manages and leads several teams of developers supporting a specific set of business applications. She has both technical and business skills and as a manager she's focused on delivering business innovation.
- She balances her time between strategic planning with her business partners, and also resolving organizational issues and roadblocks her teams are facing. She develops organizational strategies and plans to secure budget and resources for her team.

Erin the Application Development Executive (VP, etc.)

- Erin is a strategic leader focused on business challenges and the big picture.
- Her top goal is predictable Business Results

Industry Analyst Resources

Research relevant to this use case can be found in the Analyst Reports - Use Cases spreadsheet.

Market Requirements

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/d7qUiJAtQ" frameborder="0"
allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

Market Requirements is a collection of capabilities we recognize are present in the Source Code Management use case. People looking to solve this use case will consider fundamental that at least one, if not all of these requirements is present in the solution they implement. To gather these in a way that represents the market with accuracy we collect data from several sources like analysts, users, competitors and thought leaders.

Market Requirement	Description	Typical features that enable this capability	Value / ROI
--------------------	-------------	--	-------------

--	--	--	--

--- --- --- ---			
-----------------	--	--	--

Protect and secure assets	The solution provides mechanisms to host (repos) project assets (source code, designs, graphics, media, etc), place and manage different change permissions for		
---------------------------	---	--	--

the users that access those repos as well as keep a detailed chain of custody of all changes these assets are subject of. | Single sign-on, code ownership, change reviews, change approvals, IP Allowlist/Denylist, Activity stream, GPG signed commit, Reject unsigned commits, Protected branches, branching, committer access rules, Compliance dashboard etc. | Secures IP and valuable assets. Provides information on project history changes |

| Enterprise Ready | The solution is robust enough to make critical functionality available at large scale, widely distributed teams in highly regulated markets. It supports multiple project structures from monorepos to service-oriented architectures such as microservices. | Geo, Geo High Availability, Disaster Recovery, Git protocol v2 support, Deduplicate Git objects for forked repositories, Download single repository files, fast and responsive application, project templates, file templates, access controls, and traceability. | Prevents outages and disruptions of development team work. Enables traceability to authors of changes to address defects or bugs in the product and auditability throughout |

| Supports numerous assets | The solution is able to manage and maintain the version history of the diverse assets and support the development patterns that each asset implies | Component reuse, traceability, design management, branching, diffing, merging, object storage, design versioning | Able to manage assets and files for the entire development team, no matter how diverse, creating a single source of truth for the product configuration and making visibility and communication available at every level |

| Foster Collaboration | The solution is designed to enable and foster collaboration among team members. The collaboration system includes manual gates and approvals as well as automated workflows. | Create fast new branches of the project, add new files/assets, collaborate on proposed changes, review comments, suggest changes, webIDE, suggestion approvals, conflict resolution, merge, diffing, hand-offs, Design management and operations, workflow automation, Wiki, snippets, version controlled snippets, Automatically update or close related issue(s) when a merge request is merged, Configurable issue closing pattern, display merge request status for builds in CI system, visibility into security scans and build stats. | Code quality increase and improved release velocity through team review and validation. |

| Secure Development | The solution allows for security practices to be enabled at the creation phases of the project | Dependency scanning, SAST, License compliance scanning, DAST, Container scanning | Increasing resilience to external attacks, internal threats and ability to resume activity promptly |

The GitLab Solution

How GitLab Meets the Market Requirements

Market Requirements	How GitLab Delivers	GitLab Stage/Category	Demos

| Protect and secure assets | GitLab has built in access control: LDAP, Active Directory, SAML, SSO. In-app access controls include user roles with different levels of access and permissions. The IP hosted in repos can be stored and accessed through repos, projects, and groups. There are mechanisms to restrict and track changes to the IP like protected branches, protected tags, push rules, MR approvals, and Codeowners. Finally, admins are able to control events and stop non compliant ones through audit events, block users, credentials inventory, License Compliance and Compliance Dashboard. | Create stage: SCM, Code Review
 Manage stage: Compliance Management
 Release stage: Release evidence | Control Changes to Product Development Assets
 Manage, Track and Maintain Access
 Blog post: How GitLab protects your IP |

| Enterprise Ready | GitLab provides capabilities to serve huge, geographically distributed teams organized into groups, subgroups, and projects with high availability requirements.

GitLab SCM supports Geo for failover scenarios. GitLab Clusters makes Git highly available for every team, hosted in gitlab.com or on prem | Manage stage: Subgroups Code Analytics
 Create stage: Source Code Management which includes- GitLab
 Enablement stage: Geo
 Disaster Recovery
 Reference architectures | Enterprise ready
 Support different system architectures and designs
 Enterprise Ready - Project Templates |

| Support numerous assets | Repositories can host numerous types of assets such as Source Code, data narratives in Jupyter notebooks for annotated, data-driven projects, and rendered markup like GitLab flavoured Markdown, Markdown, RDoc, AsciiDoc, and Org in wikis. GitLab also hosts graphic assets and support the handover communication in issues | Create stage: SCM, Design Management, Wiki
 Git LFS | Support numerous assets |

| Foster Collaboration | GitLab is a platform to collaborate on proposals. Change proposals happen in any project through merge requests. Collaboration then can happen in multiple places like in threaded conversations for clarity, or in merge request approvals where changes can be approved or rejected. To understand the proposed changes a detailed vision of the changes is found in resolving conflicts, code reviewers can comment on specific code line, suggest changes applicable in one click, or propose to use code snippets. All these actions connect the different users through plenty of in-app, chat (Slack or Discord) or email notifications |

Create stage: SCM, Code Review, WebIDE, Single file web editor, Live Preview, Snippets
 Manage stage: Code Review Analytics, Insights
 Plan stage: Issue tracking
 Verify stage: Code Quality | Foster Collaboration |

| Secure Development | GitLab incorporates several security capabilities to make development more secure even before build happens. They can detect secrets and license compliance issues, find vulnerabilities in source code or in its dependencies before merging, find vulnerabilities in running apps before merging too, and in containers too. These capabilities can even reproduce what thousand of users input in seconds to detect faults and other issues with Fuzz testing | Secure stage: SAST, DAST, Fuzz Testing, Dependency Scanning, Container Scanning, License Compliance, Secret Detection, Security Dashboard | Secure Development Also see DevSecOps resources |

Top 3 Differentiators

Differentiator	Value	Proof Point	Demos
Distributed version control	It allows for asynch, remote, collaborative work to flourish since a single copy of the complete project's history can be stored in any machine, branching is easy and powerful so almost endless workflow possibilities open in opposition to centralized VCS like Perforce or CVS. All the information different teams produce while collaborating on source code and other digital assets in GitLab can be easily analyzed, authorized and streamlined from the Merge Request with clockwork precision. This, in turn, allows for team leads to correctly implement best practice workflows like GitLab Flow	Stackoverflow's 2018 survey data states that 87% of respondents use Git (jump from 69% in 2015) instead of other centralized and distributed VCSs. Similar trend is captured in the Open Hub data. In 2019, Stack Overflow didn't even ask the question in the same survey. · Gartner's Market Guide for Software Change and Configuration Management from 2015 lays out clearly the advantages of DVCS. In 2019 Gartner assess SCM as part of Application Release Orchestration of which GitLab is a challenger as of 2019 · Google trends since 2004 compared to other DVCS and CVCSSs.	Distributed Version Control
Single Application	The ability to connect every phase of the Software Development Lifecycle in one single DevOps platform. One data layer, one design system, one set of commands to manage all the different stages of software design, development, build and deployment	General proof	

points of the single app | Single Application |
| Product Development Management | GitLab is the only product that increasingly provides collaboration functionality to Product teams that work not only with source code but also and IP, graphic assets, animations and binaries to mention a few. | Forrester's Adopt Product Management to Connect Design and Development clearly states that "Siloed Design And Dev Teams Deliver Subpar Software" | Product Development Management |

Message House

A message house contains all the pieces of content that will clearly convey GitLab's values and differentiators in each touch point with prospects and clients. Each touchpoint should be worth their while and should also be relevant in their relationship with GitLab in order to move the needle in their decision-making process. In other words, every single item of the message house is in context with each persona, connect with their intents, resonates with their inflection points (value drivers) and provides an easy narrative of the SCM use case to lay out next steps and make progress.

The message house for the use case provides a structure to describe and discuss the value and differentiators for the use case.

Discovery Questions

The sample discovery questions below are meant to provide a baseline and help you uncover opportunities when speaking with prospects or customers who are not currently using GitLab for SCM. See a more complete list of questions, provide feedback, or comment with suggestions for GitLab's SCM discovery questions and feel free to contribute!

Sample Discovery Questions

- How do your teams stay updated of updates/changes to the codebase?
- How do different members of the team keep their changes synced with the efforts of others?
- Do the software development teams keep end to end compliance? From requirement to versioned release?
- How many times do your teams integrate their code?
- How do your teams communicate?
- What do you use to allow developers to quickly create dev environments that mimic production environments as closely as possible? (infrastructure as code + innersourcing, but if they are using Docker it might be that simple)
- How well integrated is Security in your DevSecOps workflow? Are they truly integrated, or an after-thought? How does your security team see what changes are coming down the pipe, works in progress, and when things will be ready for their input? How do they quickly understand the nature of the changes and what to test? (seeing active MRs, WIP, issue boards for planning, and code/review/commits for understanding scope and nature of the change)
- How is QA integrated into your dev workflow? Do they have access to the issues and code changes in one place? Are they able to see review apps for testing?

GitHub-specific discovery Questions

- How did you implement your HA setup? How do you run the zero-downtime upgrades?

Remarks:

GitHub HA is not a real HA setup, it's more like a failover solution (with sync between 2 virtual machines, there is no bare metal solution)

Moreover, zero-downtime upgrade seems not possible

- How long have you been operational on GitHub Actions? Can you do whatever you want with GitHub Actions?

Any security issue with GitHub Actions? How do you protect your secret variables to not be displayed in the logs of the runners?

Remarks:

GitHub documentation: GitHub automatically redacts secrets printed to the log, but you should avoid printing secrets to the log intentionally.

You cannot mask variable with GitHub Actions (with GitLab you can)

- How do you scale your GitHub Actions runners? (versus the GitLab Kubernetes executor)

Do you use GitHub as a project management tool? Are you able to implement all of Scrum's artifacts? How do you manage complex user stories or related to the same topic?

Remarks:

There are no Epics within GitHub

- Are the vulnerability alerts sufficient for you? Do you need to use them in offline mode?

Remarks:

(pending verification) "security alerts for vulnerable dependencies" of GitHub need that your GitHub instance should be connected to GitHub Enterprise Cloud (no air-gapped version)

- Are you using Codespaces? If so, what for?

Remarks:

Chances are they haven't used it yet because it's in pre release. But even so, an excellent time to mention the webIDE: light weight, embedded in GitLab's GUI, configurable, linting.

If they mention the Super Linter recently released in GitHub you can mention our CodeQuality feature that supports almost as many languages as Super Linter and the fact that Super Linter can be run without much hassle in GitLab

Remarks:

GitHub Enterprise provides Organization level Analytics (Insights) on a separate self-managed server and requires the GitHub One License.

Competitive Comparison

Amongst the many competitors in the DevOps space, GitHub, Perforce, Azure DevOps, SVN are the closest competitors offering SCM capabilities. Phabricator, Gerrit, GitHub, BitBucket, Azure DevOps, Crucible, Review Board, Reviewable, CodeStream, GitLens, VS Live Share and Gitpod are the closest offering Code Review capabilities. Cloud9, Codesandbox, Repl.it, Koding, StackBlitz, Theia, Gitpod, Coder, VS Online are the closest competitors offering a webIDE. Psatebin, Blocks, Gist.io, Bitbucket Snippets, Codesandbox, JSBin, JSFiddle, Codepen are the closest offering snippets. Invision, Figma, UX Pin are the closest competitors offering Design Management capabilities.

Industry Analyst Relations (IAR) Plan

- The IAR Handbook page has been updated to reflect our plans for incorporating Use Cases into our analyst conversations.
- For details specific to each use case, and in respect of our contractual confidentiality agreements with Industry Analyst firms, our engagement plans are available to GitLab team members in the following protected document: IAR Use Case Profile and Engagement Plan.

For a list of analysts with a current understanding of GitLab's capabilities for this use case, please reach out to Analyst Relations via Slack (analyst-relations) or by submitting an issue and selecting the "AR-Analyst-Validation" template.

Proof Points - Customer Recognitions

General proof points and Customer Recognition

Quotes and reviews

Gartner Peer Insights

Gartner Peer Insights reviews constitute the subjective opinions of individual end users based on their own experiences and do not represent the views of Gartner or its affiliates. Obvious typos have been amended.

>"The software is intuitive and quite easy to use. Since many software development projects require more than one person, this makes it easy to create teams and collaborate."

>

> - Quality Engineer, Gartner Peer Review

>

>"Improves productivity of engineers by providing easy and fast ways to keep feature branches and merge them quickly and efficiently."

>

> - Engineering Manager, Gartner Peer Review

>

>"Keeps your software projects under control. Rogue developers are kept at bay via enforced review processes and pipelines."

>

> - Engineer, Gartner Peer Review

>

>"For managing git repositories it is the best product available right now in the market."

>

> - Sr. Software Engineer, Gartner Peer Review

>

>"This has really aided in our ability to automate software delivery and return wasted overhead back to the pool of resources! This is a very simple to use and fast delivery tool to assist your code pipeline."

>

> - Project Manager, Gartner Peer Review

>

>"We use this platform in our company to version our source [code] ensure they are up to date and as a backup option. It enables us build scalable and high quality products. Ease of use and compatible with most development environments."

>

> - Sr. Software Developer, Gartner Peer Review

>

>"I appreciate its ability to run limitless. It has various features like issue tracker, protected branches and merge requests, which gives very nice experience."

>

> - Sr. Software Engineer, Gartner Peer Review

>

>"GitLab is a very useful SCM. In our [organization] we have used it as a source code repository. We have extensively used branching and tags creation feature. As we work in a sprints, we have several sprint and feature branches."

>

> - Lead Developer, Gartner Peer Review

>

>"GitLab is a superb version control and collaboration [provider]."

>

> - Systems Engineer, Gartner Peer Review

>

>"Before GitLab, we used to make local copies of code or backup the code and then pass on the code through the server. But if our organization knew about GitLab from start, we would have immediately integrated with our development practises for ease of deployment."

>

> - Software Developer, Gartner Peer Review

Case Studies

ESA (European Space Agency)

- Problem Geographic separation led to software deployment that used to take weeks.
- Solution GitLab Core (SCM,CI) allow opportunities for collaboration, synergies and multiple exploitations of efforts in visible way.
- Result More than 140 groups adopted GitLab and more than 1500 software projects have been created. These range from mission control systems, onboard software for spacecraft, image processing and monitoring tools for Labs.
- Sales Segment: Enterprise

Goldman Sachs

- Problem Needed to increase developer efficiency and software quality

- Solution: GitLab Premium (CI/CD, SCM)
 - Result: Improved from 1 build every two weeks to over a 1000/day, or releasing 6 times per day per developer, and an average cycle time from branch to merge is now 30 minutes; simplified workflow and simplified administration
- All the new strategic pieces of 'software development platforms are tied into GitLab. GitLab is used as a complete ecosystem for development, source code control and reviews, builds, testing, QA, and production deployments.
- Sales Segment: Enterprise

Worldline

- Problem Worldline faced bottlenecks and lack of ownership when it was using Subversion. It took 1-2 weeks to get a source code repo.
- Solution GitLab Core (SCM)
- Result: With GitLab Core it now takes a few seconds. Within six months, over 1,000 users were active users because GitLab is so easy to use. People excited to contribute code reviews with GitLab Merge Requests. Previous code review tools had 10-20 developers using them, while Worldline currently has 3,000 active users of GitLab - an adoption rate increase of 12,000 percent.
- Sales Segment Enterprise

NorthWestern Mutual Commit San Francisco 2020: Why we chose GitLab as our Enterprise SCM Deck

- Problem Code base was fragmented and dev permissions were also complex to handle. This prevented devs from collaborating, deploying faster and fixing bugs and security holes.
- Solution GitLab Premium (SCM, CI)
- Result A full migration to their Enterprise environment was completed in 8 months. After implementation they managed to reduce friction with the ease of use of GitLab's CI.
- Sales Segment: Enterprise

References to help you close

Links to Salesforce References Note: